AI Agent 工作模式说明

> 当前本文档对应的会话模型：GPT-5.4
>
> 适用读者：具备早期大模型对话或集成经验，了解 JSON 约束输出、工具调用、MCP、SKILL 等概念，但未持续跟进近年 Agent 工程化落地路径的研发人员。

- AI Agent 工作模式说明
  - 1. 文档目的
  - 2. 先给出工程定义
  - 3. 从 Chat 到 Agent 的能力落地路径
    - 3.1 纯对话阶段
    - 3.2 结构化输出阶段
    - 3.3 工具调用阶段
    - 3.4 上下文接地阶段
    - 3.5 Agent 编排阶段
    - 3.6 记忆与持续工作阶段
    - 3.7 技能包与子 Agent 阶段
  - 4. 当前 VS Code/Copilot 环境中，Agent 大概率被配置了哪些工作流
    - 4.1 先汇报，再深入操作
    - 4.2 先取证，再下笔
    - 4.3 多步骤任务显式维护计划
    - 4.4 文件编辑走受控补丁路径
    - 4.5 修改后要做校验
    - 4.6 允许写入外部记忆，但要克制
  - 5. 这次对话里，Agent 实际做到了什么
    - 5.1 工作区发现与版本判断
    - 5.2 跨格式上下文提取
    - 5.3 计划管理
    - 5.4 受控编辑
    - 5.5 结果校验
    - 5.6 外部记忆沉淀
    - 5.7 衍生文档生成
  - 6. 工具体系：当前环境中的能力边界
    - 6.1 上下文采集工具
    - 6.2 编辑工具

    - 6.3 执行工具
    - 6.4 验证工具
    - 6.5 计划与记忆工具
    - 6.6 子 Agent 工具
  - 7. MCP、宿主原生工具、SKILL、项目文档之间的关系
  - 8. 记忆体系详解
    - 8.1 用户记忆
    - 8.2 会话记忆
    - 8.3 仓库记忆
    - 8.4 三层记忆如何配合
  - 9. 可见状态与不可见状态
    - 9.1 对用户可见的状态
    - 9.2 默认不可见的状态
    - 9.3 当前环境中的现实边界
  - 10. 当前环境里可见的 SKILL 与子 Agent
    - 10.1 可见 SKILL
    - 10.2 可见子 Agent
  - 11. 如何理解“平台到底给 Agent 配置了什么”
  - 12. 实际使用建议
    - 12.1 对任务输入的建议
    - 12.2 对记忆使用的建议
    - 12.3 对可解释性的建议
    - 12.4 对能力预期的建议
  - 13. 一句话总结

1. 文档目的

本文档用于解释：在当前 VS Code + Copilot Agent 工作环境中，一个现代 AI Agent 到底由哪些层组成，它相较于早期只能对话的 Chat 具体新增了哪些能力，这些能力在当前对话中是如何被实际调用的，以及平台大概率为它配置了哪些工作流约束。

重点是把几个工程概念落到可观察行为上：

- 从纯对话模型到 Agent，中间增加了哪些关键能力层。
- 模型、Agent、工具、MCP、SKILL、记忆、工作区文档分别是什么关系。
- 在当前环境里，Copilot 为什么会读文件、改文件、跑终端、维护计划、写记忆。
- 哪些状态对用户可见，哪些状态默认不可见。
- 如何理解跨工作区用户记忆、当前会话临时记忆、仓库级记忆的作用与边界。

2. 先给出工程定义

在当前环境里，Copilot Agent 更准确的工程定义不是“更会聊天的 ChatBot”，而是：

模型能力 + 指令栈 + 上下文采集 + 工具编排 + 计划管理 + 校验闭环 的组合运行时。

可以拆成下表理解：

层	作用	当前对话中的体现
模型层	负责理解、推理、生成	负责读懂需求、比较 PRD 版本、组织文档结构
指令层	约束行为方式、输出格式、工具策略	先发进度更新、优先读上下文、编辑时用补丁、改完做校验
上下文层	提供当前任务所需事实	工作区文件、附件、当前打开文件、用户请求
工具层	让模型具备外部操作能力	读文件、搜索、终端执行、改文件、维护待办、写记忆
计划层	让多步骤任务可分解、可追踪	待办列表、阶段状态切换
记忆层	在会话外保存少量高价值信息	仓库记忆文件 cloud-desktop-facts.md
验证层	防止输出停留在“想当然”	错误检查、文本检索、结果复核

早期 ChatGPT 主要覆盖“模型层”；现代 Agent 的差异主要来自后 6 层的工程补全。

3. 从 Chat 到 Agent 的能力落地路径

如果把近年的能力演进压缩成一条工程主线，可以按下面的顺序理解。

3.1 纯对话阶段

早期形态只有：

- 用户输入自然语言
- 模型输出自然语言

此时模型具备理解和生成能力，但没有稳定的外部行动能力。即使要求它输出 JSON，也只是“按约束生成文本”，并没有真正访问外部系统。

3.2 结构化输出阶段

工程上最早可落地的一步，是要求模型按固定 JSON Schema 或字段约束输出，以便集成到业务系统中。

这一阶段解决的是：

- 输出可解析
- 输出更稳定
- 能接入工作流或下游程序

但它仍然没有解决“模型自己去读系统状态、修改文件、调用外部服务”的问题。

3.3 工具调用阶段

再往前一步，是把外部能力封装成结构化工具接口，让模型不只是“说应该怎么做”，而是可以请求执行一个动作，例如：

- 读取文件
- 搜索代码
- 调终端命令
- 调业务 API
- 写入数据库

这一阶段常见的技术形式包括：

- function calling
- tool calling
- 宿主平台内置工具接口
- 通过 MCP 暴露的外部工具或资源

这里要特别注意一个区分：

MCP 是一种工具暴露与资源协同的标准化方式，但 Agent 获得工具能力并不强制依赖 MCP。

在当前会话里，Agent 实际调用的是宿主平台直接提供的工具接口；从架构思想上看，这与“通过 MCP 暴露文件系统、搜索、终端能力”属于同一类工程方向，但不应机械地把所有工具都等同为 MCP 工具。

3.4 上下文接地阶段

只会调用工具还不够，模型还需要获得“与当前任务有关的真实上下文”。这就引出了：

- 工作区文件读取
- 代码/文档搜索
- 当前打开文件内容注入
- 附件内容注入
- Web 页面抓取
- 项目约束文档读取

这一层解决的是“回答不再只依赖聊天窗口里贴出来的文字”。

3.5 Agent 编排阶段

当模型既能读上下文，又能调用工具时，就进入了 Agent 工程化阶段。典型运行方式变成：

1. 观察当前上下文
2. 规划下一步动作
3. 调用工具收集信息或执行修改
4. 检查结果
5. 继续下一步，直到任务闭环

这已经不是单轮问答，而是一个简化版的 `observe -> plan -> act -> verify` 循环。

3.6 记忆与持续工作阶段

当任务跨多轮、多文件、多主题时，仅靠当前对话上下文会逐渐不稳定，于是工程上会引入外部记忆：

- 用户级长期偏好
- 会话级临时笔记
- 仓库级事实沉淀

这类记忆不是模型参数更新，而是外挂状态层。

3.7 技能包与子 Agent 阶段

再往后走，平台开始补充两类“可复用工作流”：

- **SKILL**：面向某类任务的工作说明书或领域流程包
- **子 Agent**：可被主 Agent 调用的专门执行单元

它们解决的是“同一种任务如何更稳定、更少歧义地执行”。

4. 当前 VS Code/Copilot 环境中，Agent 大概率被配置了哪些工作流

当前环境里，很多行为并不是模型自由发挥，而是宿主平台和上层指令共同约束的结果。从当前对话的实际表现看，至少可以明确以下工作流规则已经生效。

4.1 先汇报，再深入操作

在开始搜索、读取、编辑前，需要先给出简短进度说明，告诉用户：

- 当前理解的任务是什么
- 下一步准备做什么

这使 Agent 更像一个可协作的执行器，而不是静默黑盒。

4.2 先取证，再下笔

对于工作区任务，优先读取已有文件和上下文，而不是直接凭经验生成内容。

在本次 PRD 工作中，先后读取了：

- PRD v1 至 v3
- 业务对象关系图多个版本
- 最新需求清单的 Excel 文件
- 交接上下文与方法论文档

这一步是典型的“上下文接地”。

4.3 多步骤任务显式维护计划

复杂任务会拆成待办项并更新状态，例如：

- 读取最新需求来源
- 分析 PRD 增量差异
- 起草 PRD v4
- 校验文档一致性

这说明平台已经把“任务计划”视为 Agent 运行时的一部分，而不是完全依赖隐式推理。

4.4 文件编辑走受控补丁路径

当前环境要求文件编辑优先通过结构化补丁方式完成，而不是任意输出整段文本让用户自行粘贴。

这类约束带来的收益是：

- 变更更可控
- 对已有文件更友好
- 更利于局部修改和结果验证

4.5 修改后要做校验

编辑完成后，不应把“补丁成功应用”当成任务完成，而应继续检查：

- 文件是否报错
- 关键术语是否残留旧版本口径
- 输出是否与目标一致

这就是验证闭环。

4.6 允许写入外部记忆，但要克制

当前环境允许 Agent 写记忆，但不是把所有过程都记下来，而是只记录后续高概率复用的、已经验证过的事实。

本次对话中写入的仓库记忆文件 cloud-desktop-facts.md 就属于这一类。

5. 这次对话里，Agent 实际做到了什么

为了把“能力”落到具体行为上，可以直接回看本次对话中的动作链。

5.1 工作区发现与版本判断

调用目录和搜索工具，识别当前工作区有哪些文件，并确认：

- 最新业务对象关系图是云桌面业务对象关系图_v10.md
- 最新需求清单实际是一个 Excel 文件
- PRD 已存在 v1、v2、v3 可供对比

这一步对应的是“工作区感知能力”。

5.2 跨格式上下文提取

因为需求清单是 Excel，不适合直接当纯文本读取，所以额外调用了终端，在本地运行 Python + openpyxl 将表内容提取出来。

这一步说明 Agent 不只是“会读文本文件”，还可以通过工具链绕过格式障碍，把二进制或结构化文件转成可分析内容。

5.3 计划管理

在任务进行过程中，显式维护了待办列表和状态变化。这表明当前 Agent 不是单次生成，而是带有阶段控制的执行流。

5.4 受控编辑

通过结构化补丁创建了云桌面PRD_v4.md，而不是仅输出一大段文本。

5.5 结果校验

创建 PRD 后，又继续执行：

- 错误检查
- 关键术语检索
- 一致性复核

5.6 外部记忆沉淀

把本仓库后续高频会复用的事实写入了仓库记忆 cloud-desktop-facts.md，用于减少后续轮次再次回到旧术语体系的概率。

5.7 衍生文档生成

在主线任务之外，又继续创建了本文档，用于解释 Agent 工作模式本身。

从工程视角看，这一整串动作已经明显超出“纯 Chat 回答”的范畴，属于典型的 IDE 内 Agent 工作流。

6. 工具体系：当前环境中的能力边界

当前会话可见的工具类型大致可以分为 6 类。

6.1 上下文采集工具

用于确定事实来源和范围，例如：

- 列目录
- 搜文件
- 全文检索
- 读文件
- 读附件或网页

6.2 编辑工具

用于新增或修改工作区文件，例如：

- 结构化补丁编辑
- 创建文件
- 创建目录

6.3 执行工具

用于补齐模型自身无法直接完成的外部操作，例如：

- 跑终端命令
- 调任务
- 读取终端输出

6.4 验证工具

用于在动作之后核查结果，例如：

- 获取错误信息
- 再次检索关键内容
- 查看 git 变化

6.5 计划与记忆工具

用于维持多步骤任务的稳定性，例如：

- 更新待办列表
- 写入记忆
- 查看记忆

6.6 子 Agent 工具

用于把探索类工作外包给更聚焦的执行单元。

当前会话可见的子 Agent 是 Explore，定位是只读探索助手。

7. MCP、宿主原生工具、SKILL、项目文档之间的关系

这些概念经常被混在一起，实际应分开理解。

概念	本质	解决的问题
模型	语言理解与生成引擎	能不能理解与推理
Agent	带运行时编排的执行器	能不能把多步任务闭环完成
工具	可调用的外部动作接口	能不能真正读、写、查、跑
MCP	工具/资源暴露协议的一种标准化方式	工具如何被统一接入和描述
SKILL	某类任务的工作说明书	相似任务如何更稳定执行
项目文档	当前任务的事实来源	模型应该基于什么说话
记忆	会话外保留的少量高价值状态	下一轮如何延续已确认口径

一个常见误区是把 SKILL 理解成“模型学会了一个新能力”。更准确的理解是：

SKILL 是附加工作流，不是神经网络参数层面的新技能。

同理，MCP 也不是“让模型突然更聪明”，而是让模型可以更标准化地接入外部工具和资源。

8. 记忆体系详解

当前环境里的记忆明确分成 3 层。这一层非常关键，因为它直接决定 Agent 是否能跨轮次稳定工作。

8.1 用户记忆

位置：/memories/

生命周期：跨工作区、跨会话保留。

适合存什么：

- 长期稳定的写作偏好
- 固定的输出结构偏好
- 常用技术栈偏好
- 经常重复出现的工作习惯

不适合存什么：

- 某个项目的一次性背景
- 很长的大段原文
- 仍未确认的结论

如何设置：

可以显式要求 Agent 将某条长期偏好记录为用户记忆。工程上，它会表现为在 /memories/ 根目录下写入一个简洁的记忆文件，后续在其他工作区和其他会话中也可能继续生效。

示例用途：

- 默认输出偏精炼
- 默认先给结论再给细节
- 默认对前端设计任务保持现有设计系统风格

8.2 会话记忆

位置：/memories/session/

生命周期：仅当前对话有效，对话结束后清空。

适合存什么：

- 当前任务的中间结论
- 临时假设
- 长任务里的阶段摘要
- 需要在本轮后续步骤继续引用的工作状态

如何设置：

可以要求 Agent 把某个临时工作状态写到会话记忆中。它适合作为“当前任务的临时便签”，而不是长期配置。

适用场景：

- 多轮迭代中记录“待确认问题清单”

- 保存“本轮已排除的方案”
- 记录正在进行的大型重构的阶段性判断

8.3 仓库记忆

位置：/memories/repo/

生命周期：面向当前仓库或当前工作主题。

适合存什么：

- 已验证的构建命令
- 代码库约定
- 已确认的术语边界
- 高复用事实源

本次对话中的实例：

- 最新功能事实源来自哪个文件
- 最新概念边界来自哪个文件
- “桌面/快照”如何对外表达

如何设置：

可以要求 Agent 将某条仓库事实写入仓库记忆。与用户记忆相比，它不应承载跨项目的偏好；与会话记忆相比，它更强调“已验证、值得复用”。

在当前这类环境中，仓库记忆的写入策略通常比用户记忆和会话记忆更保守，更适合以新增事实条目的方式沉淀结论，而不是频繁重写为大段工作日志。

8.4 三层记忆如何配合

可以把三层记忆理解为：

- **用户记忆**：长期偏好
- **会话记忆**：当前任务临时状态
- **仓库记忆**：当前项目的稳定事实

一个简单判断准则是：

- 离开当前项目还成立，用用户记忆
- 只在当前会话里有意义，用会话记忆
- 只对当前仓库长期有用，用仓库记忆

9. 可见状态与不可见状态

从研发视角，最容易问的问题不是“模型想了什么”，而是“运行时暴露了哪些状态”。

9.1 对用户可见的状态

当前环境中，通常可见：

- 进度说明
- 工具调用及其返回结果
- 文件变更结果
- 待办状态更新

9.2 默认不可见的状态

默认不可见的通常包括：

- 完整内部推理链
- 极细粒度的候选方案比较过程
- 精确的 token 级上下文计量信息

这不是某个“被藏起来的 CLI 开关”那么简单，而是运行时本来就把“内部分析层”和“对外展示层”分开设计。

9.3 当前环境中的现实边界

在本次环境里，可以合理认为：

- 有内部分析层
- 有用户可见的进度与工具结果层
- 没有一个明确暴露给用户的“完整思维链显示开关”
- 没有一个精确对外公开的实时上下文容量仪表盘

因此，工程上更可行的诉求不是“要求展示完整内部思维链条”，而是要求展示：

- 决策依据摘要
- 当前计划
- 关键风险
- 已执行动作
- 校验结果

10. 当前环境里可见的 SKILL 与子 Agent

在本次会话中，显式可见的附加资源其实并不多。

10.1 可见 SKILL

当前可见的 SKILL 为 agent-customization。

它服务的是“Agent 自定义相关文件”的创建、修复、诊断，而不是一般业务文档编写。

这说明：

- SKILL 是按任务域挂载的
- 不是所有任务都会触发 SKILL
- Agent 需要判断某个 SKILL 是否与当前任务相关

10.2 可见子 Agent

当前可见的子 Agent 为 Explore。

它的定位是：在需要快速只读探索工作区时，替主 Agent 做更聚焦的搜索和定位工作。

这说明现代 Agent 运行时已经开始具备轻量级“分工”能力，但这种分工仍由宿主平台提供，并不等于一个完全自治的多智能体系统。

11. 如何理解“平台到底给 Agent 配置了什么”

如果从工程落地角度反推，当前这类 IDE Agent 平台通常至少完成了以下配置：

1. **模型接入**：指定基础模型，例如本次会话中的 GPT-5.4。
2. **指令编排**：定义系统级、开发者级、宿主级行为规则。
3. **上下文注入**：把工作区结构、附件、当前文件、用户请求等注入到会话中。
4. **工具注册**：向模型暴露文件系统、搜索、终端、记忆、计划等能力。
5. **权限控制**：限制哪些操作可执行，哪些操作必须谨慎，哪些路径可写。
6. **结果回显**：把工具调用结果和文件变更回显给用户。
7. **安全边界**：限制高风险内容、危险命令、敏感行为。
8. **工作流模板**：要求 Agent 先读上下文、先汇报、再编辑、编辑后验证。

也就是说，用户看到的“Agent 很会做事”，很大一部分来自平台工程，而不只是模型本体。

12. 实际使用建议

12.1 对任务输入的建议

如果希望 Agent 输出更稳，建议优先提供：

- 最新事实源文件
- 概念边界文档
- 已确认术语表
- 不希望越界的范围说明
- 需要保留的风格与结构偏好

12.2 对记忆使用的建议

建议显式区分：

- 需要跨项目保留的偏好，写入用户记忆
- 只在当前任务临时有效的状态，写入会话记忆
- 当前项目长期复用的事实，写入仓库记忆

不要把未经验证的猜测直接写进仓库记忆，也不要把冗长原文整段塞进长期记忆。

12.3 对可解释性的建议

对于这类 Agent，比较有效的要求方式是：

- 说明依据
- 说明计划
- 说明为什么调用某个工具
- 说明改动结果和验证结果

不建议把“完整内部思维链条”当作默认应暴露的信息。

12.4 对能力预期的建议

应把这类 Agent 看作“受控执行器”，而不是“自动理解一切的全知系统”。

它非常适合：

- 读工作区资料并汇总
- 起草和修改文档

- 搜索与比较多个版本
- 调用工具完成局部自动化
- 对结果做基础校验

但它仍然依赖：

- 明确的事实源
- 清晰的术语边界
- 足够好的工具暴露
- 合理的权限和验证策略

13. 一句话总结

当前 VS Code + Copilot 环境中的 AI Agent，可以理解为：

一个由模型、指令、上下文、工具、记忆和验证闭环共同构成的 IDE 内执行运行时；它相较于早期 Chat 的关键进步，不在于“更会说”，而在于“能够基于真实工作区上下文执行多步任务并验证结果”。